OOPS (PPF) MINI-PROJECT

COMPLEX NUMBER CALCULATOR

A C++ program using classes, objects, constructors, encapsulation and operator overloading

Submitted by	Aditi Mishra
Roll Number	2556421
Program / Section	B.Tech CSE – B
University	CGC University
Subject	OOPS (PPF)
Submitted to	Ms. Yamini Sharma

Mini-Project 1

1. Introduction

A complex number has two parts: a real part and an imaginary part. It is commonly written as a + bi, where a is the real part, b is the imaginary part, and i represents the imaginary unit. This project performs addition and subtraction on two complex numbers.

2. Objectives

• Create a Complex class to store real and imaginary parts.
• Add and subtract two complex numbers using overloaded operators.
• Display complex numbers using the overloaded << operator.
• Demonstrate basic Object-Oriented Programming concepts in C++.

3. OOPS Concepts Used

Class: Complex is a user-defined class that groups data and functions.
Object: c1 and c2 are objects representing the two complex numbers.
Constructor: Initializes the real and imaginary parts when an object is created.
Encapsulation: The data members real and imag are private and are accessed through class operations.
Operator overloading: + and - are overloaded for complex-number arithmetic. The << operator is overloaded to display a complex number.

4. Working / Algorithm

1. Start the program.
2. Read the real and imaginary parts of two complex numbers.
3. Create two Complex objects using the constructor.
4. Use the overloaded + operator to calculate the sum.
5. Use the overloaded - operator to calculate the difference.
6. Display both numbers and the results using the overloaded << operator.
7. Stop the program.

5. Source Code (C++)

Save the following code in a file named **complex_calculator.cpp**.

```
#include <iostream>
using namespace std;

class Complex {
private:
    double real, imag;

public:
    Complex(double r = 0, double i = 0) {
        real = r;
        imag = i;
    }

    Complex operator+(const Complex& other) const {
        return Complex(real + other.real, imag + other.imag);
    }

    Complex operator-(const Complex& other) const {
        return Complex(real - other.real, imag - other.imag);
    }

    friend ostream& operator<<(ostream& out, const Complex& c) {
        out << c.real;
        if (c.imag >= 0)
            out << " + " << c.imag << "i";
```

```
        else
            out << " - " << -c.imag << "i";
        return out;
    }
};

int main() {
    double r1, i1, r2, i2;

    cout << "COMPLEX NUMBER CALCULATOR\\n";
    cout << "Enter real and imaginary parts of first number: ";
    if (!(cin >> r1 >> i1)) {
        cout << "Invalid input. Please enter numeric values.\\n";
        return 1;
    }

    cout << "Enter real and imaginary parts of second number: ";
    if (!(cin >> r2 >> i2)) {
        cout << "Invalid input. Please enter numeric values.\\n";
        return 1;
    }

    Complex c1(r1, i1), c2(r2, i2);
    Complex sum = c1 + c2;
    Complex difference = c1 - c2;

    cout << "\\nFirst number: " << c1 << endl;
    cout << "Second number: " << c2 << endl;
    cout << "Addition: " << sum << endl;
    cout << "Subtraction: " << difference << endl;

    return 0;
}
```

6. Sample Output

Example input: first number = 4 + 3i, second number = 2 + 5i

```
COMPLEX NUMBER CALCULATOR
Enter real and imaginary parts of first number: 4 3
Enter real and imaginary parts of second number: 2 5

First number: 4 + 3i
Second number: 2 + 5i
Addition: 6 + 8i
Subtraction: 2 - 2i
```

7. Test Cases

First number	Second number	Addition	Subtraction
4 + 3i	2 + 5i	6 + 8i	2 - 2i
1 + 2i	1 + 2i	2 + 4i	0 + 0i
5 + 0i	2 + 0i	7 + 0i	3 + 0i

8. Input Validation

The program checks whether the user enters numeric values for all four parts. If the input is not numeric, it displays an error message and exits instead of performing calculations.

9. Conclusion

The Complex Number Calculator demonstrates how C++ operator overloading can make arithmetic operations on user-defined objects clear and convenient. It also demonstrates classes, objects, constructors and encapsulation.

10. How to Compile and Run

Using a C++ compiler such as g++:

```
g++ complex_calculator.cpp -o complex_calculator
./complex_calculator
```

On Windows, run the generated executable (for example, `complex_calculator.exe`).

Before submission: run the program on your own computer and add your own genuine screenshot if your teacher requires one.